RMIT UNIVERSITY

Report

Robot Controller with esp32

Dinh-Son Vu

Table of Contents

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This is NOT a simulation. Gazebo is no longer used. The hardware is required.
- VSCode and its extension PlatformIO (PIO) should be installed.
- First, connect the ESP32 to the computer to test the communication between ROS2 and the hardware. Later, it would be mandatory to connect the RPI to the ESP32, and ssh into the RPI.

2. Tasks

It is now time to use the real hardware. So far, Gazebo was used to getting comfortable with the ROS2 environment and packages. Gazebo was in charge of broadcasting the joint_state. Now the hardware package, included in this lesson, listens to the serial communication (esp32) and publishes the joint_state. Note that the ESP32 firmware must be flashed on your ESP32 microcontroller. It can be found in the hardware package in the following folder:
src/rmitbot_firmware/arduino_firmware/rmitbot_arduino_firmware
Please follow the guidelines relative to VSCode, PIO for flashing software:
https://randomnerdtutorials.com/vs-code-platformio-ide-esp32-esp8266-arduino/

2.1. With the esp32 connected to the computer

Tasks	Code
Install serial communication library	sudo apt install libserial-dev
Clone the repo from github	git clone https://github.com/ROS2-rmitbot/lesson7_ws.git
Navigate to the workspace, build and source it	cd ~/lesson7_ws rm -rf build install log colcon build source install/setup.bash code .
Launch the bringup launch	ros2 launch rmitbot_bringup rmitbot.launch.py

Use the teleopkeyboard to move the robot around and verify that the real robot and the robot in rviz are moving in a similar fashion. You may add and visualize the imu topic in rviz as well. Its integration with robot_localization (ekf) is performed in the next lesson.

2.2. With the esp32 connected to the RPI

So far, the computer is connected to the ESP32. A more common setup is as shown in Figure 1. Usually to ssh into your RPI, you may open a terminal and enter the command:

ssh username@ipaddr.

However, one may use the VSCode ssh capabilities. Please look at the following link:

https://youtu.be/jvi1nmKK81Y?si=ASWLSq4Oun-zVKg9

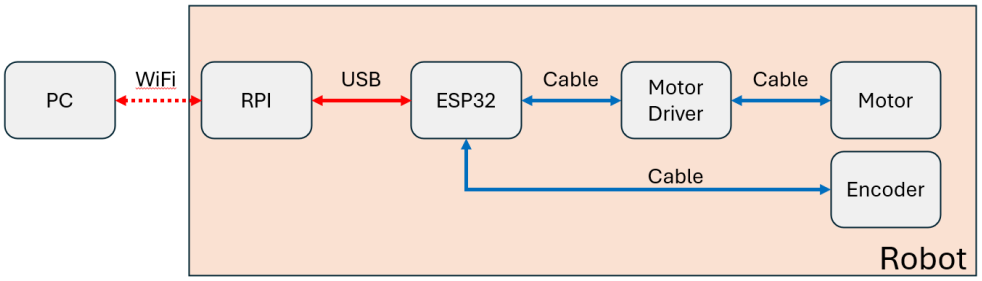

Figure 1: Robot configuration and connection

Tasks	Code
Start SSH **(make sure the RPI and PC are connected on the same WiFi)**	Open terminal → ssh username@ipaddr
On the RPI **Install serial communication library**	sudo apt install libserial-dev
On the RPI **Clone the repo from github**	git clone https://github.com/ROS2-rmitbot/lesson7_ws.git
On the RPI **Navigate to the workspace, build and source it**	cd ~/lesson7_ws rm -rf build install log colcon build source install/setup.bash
On the RPI **Launch the controller**	ros2 launch rmit_controller controller.launch.py
On the PC **Open a new terminal** **Start rviz2**	ros2 launch rmitbot_description display.launch.py
On the PC **Start teleopkeyboard**	ros2 launch rmitbot_controller teleopkeyboard.launch.py

You should be able to control the real robot from the teleoperation keyboard of the PC.

2.3 Clock between PC and RPI

You may experience some delay between the RPI and the PC. On Wi-Fi, the ping should be around 10ms. However, if the clock is not synchronised between the two devices, more important lag may be experienced.

Tasks	Code
Check if the ping is fine with the rpi	ping ipaddr
Install chrony on each device	sudo apt install chrony -y
Check status	timedatectl Normally, you should have: System clock synchronized: yes NTP service: active

3. Explanation

There are few modifications that have been performed in the packages:

- use_sim_time parameter has been set to false in all packages
- The urdf is calling the hardware interface instead of the gazebo plugin.
- For the moment, only the hardware connected to the esp32 is tested. The integration of the Lidar and camera is covered in the next packages.

4. Packages, nodes, and topics

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot
~~**rmitbot_localization**~~	~~ekf~~	~~Launch the sensor fusion between wheel encoder and imu~~
~~**rmitbot_mapping**~~	~~slam~~	~~Launch the lidar and slam~~
~~**rmitbot_navigation**~~	~~nav2~~	~~Launch the navigation~~
~~**rmitbot_vision**~~	~~apriltag~~	~~Include the plugin for camera. Launch the apriltag detection~~
rmitbot_firmware	Arduino firmware Hardware interface	The arduino firmware is the code flashed on the esp32. The hardware interface is the node launched with ROS2.

5. Future works

Verify that your real hardware robot is moving in the same fashion as your robot in rviz. If there is discrepancy, could you measure it?